Rapport de Laboratoire 10 — SEG 3502

Conteneurs — Docker, Docker Compose et Kubernetes pour l'application BookStore

Étudiant 1 : Anwar `N° 300254189`

Étudiant 2 : Manasse `N° 300268472`

Cours : SEG 3502 (Conception de logiciels distribués)

Date : Octobre 2026 | **Projet :** `books-rest-api + bookstore-client`

Dépôt GitHub du projet : https://github.com/Anwarito-lbd/SEG3502-Lab10.git

1. Brève Description du Travail Réalisé

Ce dixième laboratoire porte sur la **conteneurisation** et l'**orchestration** d'une application distribuée. L'application BookStore (API Spring Boot + client Angular + MySQL) a été conteneurisée :

- **Docker :** `Dockerfile` multi-étapes pour l'API (`books-rest-api:latest` , configuration MySQL par variables d'environnement) et pour le client (`books-api-client:latest` , nginx; URL de l'API injectée au démarrage par `entrypoint.sh` et `env.template.json`). Réseau `mysql-network` et conteneurs `mysqldb` , `booksserver` , `booksclient` (Q1, Q2).
- **Docker Compose :** `docker-compose.yml` avec les services `mysql` (volume, healthcheck), `booksserver` et `booksapp` (Q3).
- **Kubernetes :** déploiements et services `mysql` , `booksserver` (2 répliques), `booksapp` (2 répliques), PersistentVolume et PersistentVolumeClaim (Q4–Q6), exécutés sur un cluster kind.

Note : dans l'environnement Linux utilisé pour les captures, le pare-feu iptables de Docker n'était pas disponible; les conteneurs Docker y ont été lancés en mode réseau hôte (mêmes images et variables). Les commandes standard sont données à la section 4.

2. Captures d'Écran de l'Application

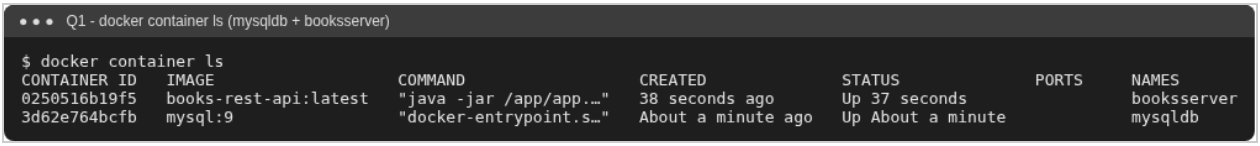


Figure 1 : Q1 — docker container ls : conteneurs mysqldb et booksserver en exécution.

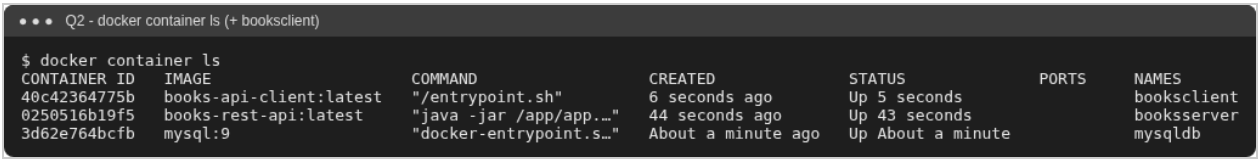


Figure 2 : Q2 — docker container ls après l'ajout du conteneur booksclient.

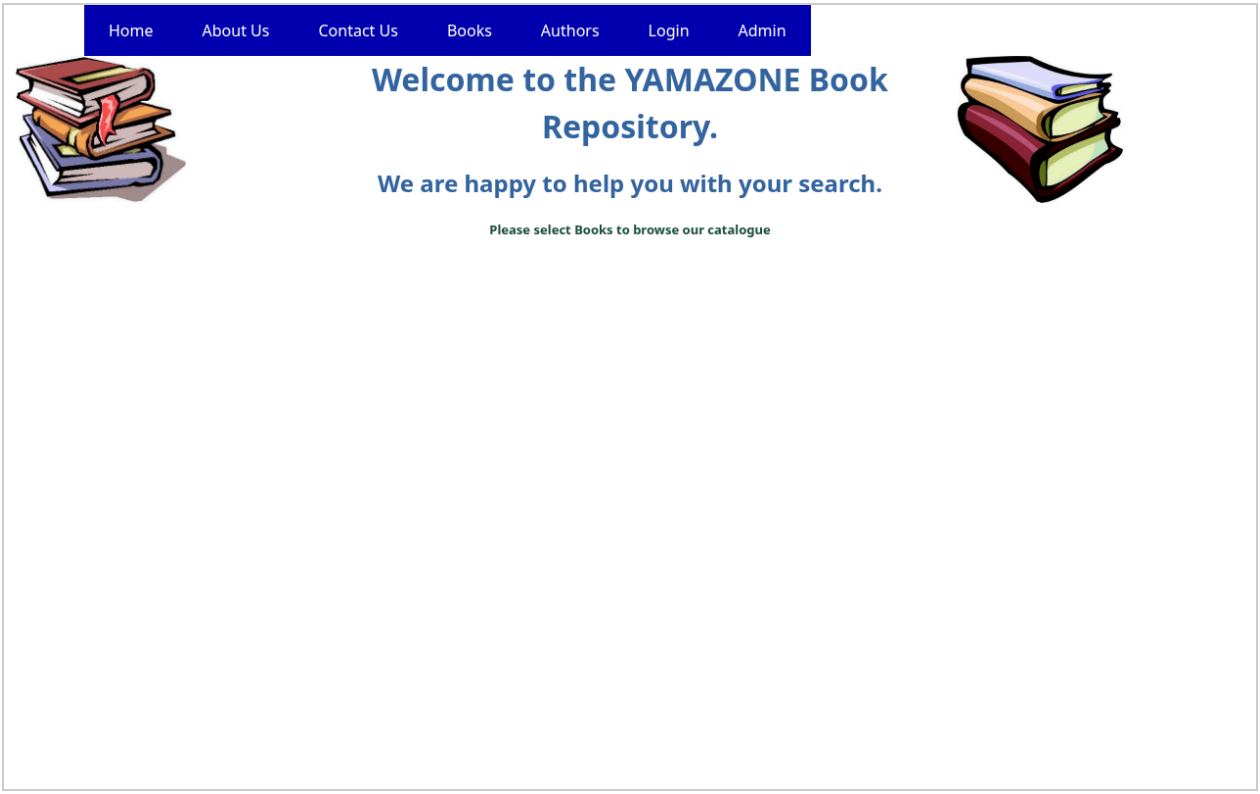


Figure 3 : Client BookStore servi par le conteneur nginx (books-api-client).

Q3 - docker compose up -d --build

```
$ docker compose up -d --build
#14 exporting layers 0.1s done
#14 naming to docker.io/library/books-rest-api:latest done
#34 exporting layers 0.0s done
#34 naming to docker.io/library/books-api-client:latest done
 Volume "docker_mysql-data"  Creating
 Volume "docker_mysql-data"  Created
 Container docker-mysql-1  Creating
 Container docker-mysql-1  Created
 Container docker-booksserver-1  Creating
 Container docker-booksserver-1  Created
 Container docker-booksapp-1  Creating
 Container docker-booksapp-1  Created
 Container docker-mysql-1  Starting
 Container docker-mysql-1  Started
 Container docker-mysql-1  Waiting
 Container docker-mysql-1  Healthy
 Container docker-booksserver-1  Starting
 Container docker-booksserver-1  Started
 Container docker-booksapp-1  Starting
 Container docker-booksapp-1  Started
$ docker compose ps
NAME                IMAGE                       STATUS
docker-booksapp-1      books-api-client:latest   Up 36 seconds
docker-booksserver-1   books-rest-api:latest     Up 36 seconds
docker-mysql-1         mysql:9                   Up About a minute (healthy)
```

Figure 4 : Q3 — docker compose up -d --build : construction des images et démarrage des 3 services.

Q4 - kubectl apply -f .

```
$ kubectl apply -f .
service/booksapp created
deployment.apps/booksapp created
service/booksserver created
deployment.apps/booksserver created
service/mysql created
deployment.apps/mysql created
persistentvolume/mysql-data created
persistentvolumeclaim/mysql-data created
```

Figure 5 : Q4 — kubectl apply -f . : création des services, déploiements et volumes.

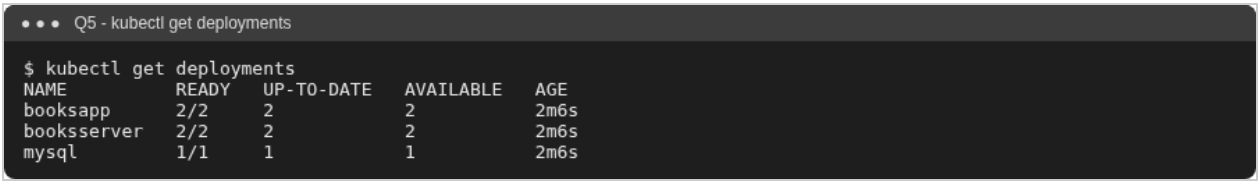


Figure 6 : Q5 — kubectl get deployments : booksapp 2/2, booksserver 2/2, mysql 1/1.

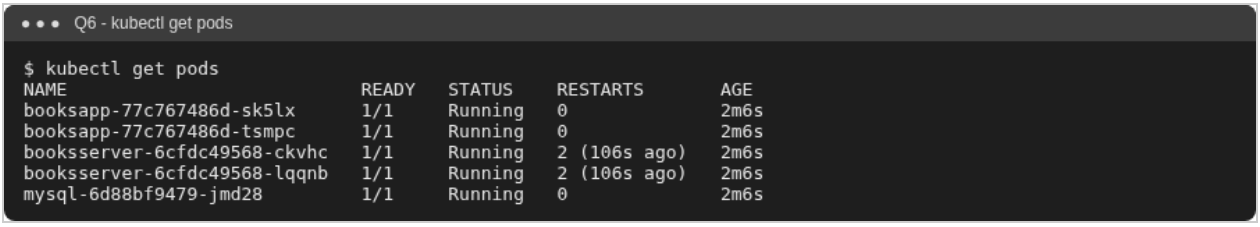


Figure 7 : Q6 — kubectl get pods : les 5 pods sont Running.

3. Validation et Tests

Test / Requête	Vérification	Résultat obtenu	Statut
`docker build (API et client)`	Images books-rest-api et books-api-client construites	Succès	Succès
`GET /books-api/authors (conteneur)`	API connectée à MySQL	HTTP 200	Succès
`http://localhost (booksclient)`	Client Angular servi par nginx, env.json généré	Succès	Succès
`docker compose up -d --build`	mysql healthy, booksserver et booksapp démarrés	Succès	Succès
`kubectl get deployments`	Toutes les répliques disponibles	5 / 5	Succès
`kubectl port-forward svc/booksserver`	API accessible dans Kubernetes	HTTP 200	Succès
Total	Ensemble des fonctionnalités du lab 10	6 / 6	**100 % réussi**

4. Étapes Nécessaires pour Installer et Exécuter le Projet

```
# Prérequis : Docker Desktop (Kubernetes activé) ou Docker + kind/minikube, kubectl
git clone https://github.com/Anwarito-lbd/SEG3502-Lab10.git
cd SEG3502-Lab10

# 1. Docker (Q1, Q2)
docker build -t books-rest-api:latest books-rest-api
docker build -t books-api-client:latest bookstore-client
docker network create mysql-network
docker run -d --network mysql-network --name mysqldb -e MYSQL_ROOT_PASSWORD=root -e
MYSQL_DATABASE=booksDb mysql:9
docker run -d --network mysql-network --name booksserver -p 8080:8080 -e MYSQL_USER=root   -e
MYSQL_PASSWORD=root -e MYSQL_DB=booksDb -e MYSQL_HOST=mysqldb books-rest-api:latest
docker run -d --name booksclient -p 4100:80 -e API_URL=http://localhost:8080/books-api/ books-api-
client:latest
docker container ls                    # application : http://localhost:4100
docker rm -f booksclient booksserver mysqldb

# 2. Docker Compose (Q3)
cd docker && docker compose up -d --build && cd ..
```

```
# 3. Kubernetes (Q4-Q6) — avec kind : kind load docker-image mysql:8 books-rest-api:latest books-
api-client:latest
cd k8s
kubectl apply -f .
kubectl get deployments
kubectl get pods
```

5. Conclusion

Le laboratoire 10 est complet : l'application BookStore fonctionne en conteneurs Docker, est démarrée en une commande avec Docker Compose et est déployée sur Kubernetes avec plusieurs répliques.